

DOCUMENTO DE INVESTIGACIÓN

La Estrategia de Lanchas Rápidas

Una exploración sobre cómo integrar inteligencia artificial en el trabajo de consultoría, con un enfoque liviano y sostenible.

Documento de investigación — Borrador para discusión interna
Mayo 2026

Resumen Ejecutivo

La idea central

Mientras la industria debate cómo construir sistemas de IA cada vez más complejos, un patrón distinto está ganando terreno: arquitecturas livianas que conectan modelos de lenguaje directamente con las fuentes de datos que necesitan, sin intermediarios.

Este documento explora si ese patrón — que llamamos informalmente la estrategia de las lanchas rápidas — puede aplicarse al trabajo de consultoría para que los equipos tengan el contexto correcto en el momento correcto, sin depender de procesos manuales ni de infraestructura difícil de sostener.

Es una hipótesis en construcción, no una propuesta cerrada.

Este documento no contiene estimados de costo ni cronogramas definitivos. Su propósito es articular la dirección estratégica y abrir la conversación sobre si este es el camino correcto antes de avanzar hacia decisiones de implementación.

Parte I: El Problema que Queremos Resolver

1.1 El costo de no tener el contexto correcto

La mayor fricción que enfrenta un consultor al producir un entregable no es técnica: es de contexto. Antes de poder escribir un BRSD, un reporte de estado o preparar una reunión, hay que ir a buscar información dispersa — en correos, en tickets de proyecto, en notas de reuniones pasadas, en documentos en Drive.

Ese proceso de recopilación es manual, consume tiempo valioso, y con frecuencia resulta incompleto. La consecuencia no es solo ineficiencia: es que los entregables se producen con menos contexto del que existe, lo que se traduce en revisiones evitables y en conocimiento institucional que no se aprovecha.

La firma ya cuenta con toda esa información. El problema es que no está conectada al momento en que se necesita.

1.2 Las herramientas actuales y sus límites

Se han explorado varias alternativas. Cada una resuelve parte del problema, pero ninguna lo resuelve de raíz:

Herramienta	Lo que resuelve	El límite
NotebookLM	Análisis con contexto cargado	El contexto hay que subirlo a mano. No escala.
Claude con integraciones nativas	Acceso a correos y tickets del usuario	Solo ve la cuenta individual. Sin visión de proyecto.
Claude Projects	Instrucciones persistentes por cliente	Contexto estático. No trae datos vivos.

El patrón que surge de este análisis es claro: las herramientas disponibles son demasiado manuales para escalar, o demasiado dependientes de infraestructura compleja para sostenerse. La dirección que vale la pena explorar está en el medio.

Parte II: La Metáfora de las Lanchas Rápidas

2.1 El gran navío

Existe en la firma un desarrollo interno de gran ambición — un sistema de orquestación de agentes de IA, capaz de conectar múltiples fuentes de datos, razonar sobre ellas y producir respuestas complejas. Es un proyecto que apunta alto y que, cuando esté listo, puede ser extraordinario.

Pero construir algo así lleva tiempo. Y mientras se construye, el mar no espera.

La industria de la IA se está moviendo con una velocidad que hace que los desarrollos largos y monolíticos corran el riesgo de quedar obsoletos antes de salir al agua. No porque la idea sea mala, sino porque el terreno se mueve debajo de los pies más rápido de lo que cualquier arquitectura interna puede seguir.

2.2 Los piratas y la lección de la agilidad

Los piratas somalíes que operan en el Océano Índico no usan fragatas. Usan lanchas rápidas. No pretenden hundir los buques que abordan: solo necesitan moverse más rápido y llegar primero. La ventaja no está en el tamaño sino en la velocidad de respuesta.

Algo similar está ocurriendo en el mundo de la IA aplicada. Las empresas más ágiles no son las que tienen el sistema más sofisticado — son las que tienen algo funcionando hoy, con lo que existe hoy, y que saben adaptarse cuando lo que existe mejora mañana.

Eso es lo que esta estrategia propone explorar.

La tesis en una sola idea

El gran navío sigue en construcción. Seguirá. Y cuando esté listo, será valioso.

Pero mientras tanto, queremos una lancha rápida: algo que salga al mar pronto, que funcione con lo que la IA ofrece hoy, y que mejore automáticamente cuando la IA mejore — sin necesidad de reconstruir nada.

El barco grande sigue en construcción. Pero el mar no espera.

Parte III: La Dirección Técnica — ¿Qué son las lanchas rápidas?

3.1 Un nuevo estándar en la industria

En noviembre de 2024, Anthropic publicó un estándar abierto llamado Model Context Protocol (MCP). La idea es sencilla de entender aunque profunda en sus implicaciones: define una forma estandarizada de conectar un modelo de lenguaje con fuentes de datos y herramientas externas.

Lo relevante no es el nombre técnico. Lo relevante es lo que hace posible: que Claude, en lugar de operar solo con lo que el usuario escribe, pueda ir a buscar información real — tickets de Wrike, reuniones, correos, documentos — en el momento en que la necesita, sin que nadie tenga que subirla manualmente.

Desde su publicación, el estándar fue adoptado por OpenAI, Google, Microsoft y Amazon, y fue donado a la Linux Foundation. Hoy existe un ecosistema creciente de herramientas que lo implementan. No parece ser una moda — parece ser la infraestructura sobre la que se construirá la siguiente generación de aplicaciones de IA.

3.2 La diferencia de filosofía

Existe una distinción fundamental entre dos formas de integrar IA en un proceso de negocio:

Orquestación propia	Delegación al modelo
✗ El código decide qué buscar y cuándo ✗ El razonamiento vive en la aplicación ✗ Cada mejora del modelo requiere adaptar el código ✗ Complejidad crece con cada caso de uso nuevo ✗ Difícil de mantener cuando el equipo crece	✓ Claude decide qué buscar y cuándo ✓ El razonamiento vive en el modelo ✓ Cada mejora de Claude mejora la solución sola ✓ La complejidad queda del lado del proveedor ✓ La interfaz es Claude — ya conocida por el equipo

La estrategia de las lanchas rápidas apuesta por el segundo camino. No porque el primero sea incorrecto — puede ser el adecuado a largo plazo — sino porque el segundo puede funcionar hoy, con menos riesgo, y evoluciona solo.

3.3 Tres capas, no más

La arquitectura que se propone explorar es intencionalmente simple:

Capa 1	Claude	El modelo de lenguaje actúa como cerebro y como interfaz. No hay orquestador custom — Claude decide qué herramientas usar, cuándo usarlas y cómo sintetizar los resultados. Los consultores interactúan con Claude de la misma forma en que ya lo hacen hoy.
Capa 2	Servidor de herramientas (MCP)	Un componente liviano que expone un conjunto pequeño de herramientas de acceso a datos: traer tickets de un proyecto, obtener transcripts de reuniones, buscar correos relevantes, listar documentos. No razona ni sintetiza — solo abre puertas y devuelve información.
Capa 3	Fuentes de datos existentes	Wrike, Google Workspace (Gmail y Drive), Zoom. La información ya existe en estos sistemas. El objetivo no es copiarla a otro lado, sino darle a Claude acceso directo cuando lo necesita.

Parte IV: Cómo Cambia el Trabajo del Consultor

4.1 La experiencia en la práctica

El consultor abre Claude en su navegador — sin nueva aplicación, sin manual de usuario. Selecciona el Proyecto correspondiente al cliente en el que está trabajando. Ese Proyecto ya tiene cargadas las instrucciones del proceso, las plantillas de la firma y los criterios de calidad del entregable que va a producir.

Escribe en lenguaje natural lo que necesita. Claude usa las herramientas disponibles para traer el contexto real del proyecto — tickets actualizados, lo que se discutió en las reuniones recientes, correos relevantes del cliente — y lo sintetiza. Hace preguntas cuando falta información. Guía el proceso paso a paso. El consultor nunca sale de Claude.

La diferencia no es solo de velocidad. Es de profundidad: un consultor que llega a cada entregable con el contexto completo del proyecto produce trabajo más informado, con menos riesgo de omitir algo que ya se había definido.

4.2 Casos de uso que se pueden explorar

Los siguientes son ejemplos de situaciones donde este enfoque podría generar valor inmediato. No son todos los casos posibles ni una lista definitiva — son puntos de partida para la conversación:

Entregable o situación	Lo que haría Claude	Contexto que traería automáticamente
BRSD / FSD	Guiar el proceso, estructurar el documento, hacer preguntas de calidad, retroalimentar cada sección	Reuniones de levantamiento, tickets del proyecto, documentos previos del cliente
Weekly Status Report	Consolidar actividades del período, generar el reporte, facilitar la revisión	Tickets actualizados, reuniones recientes, correos del cliente
Preparación de reunión	Resumir el estado del proyecto, identificar puntos abiertos, sugerir preguntas clave	Última reunión, tickets sin resolver, correos recientes
Onboarding a proyecto	Explicar el historial, los acuerdos tomados, los actores relevantes	Todas las reuniones del proyecto, todos los tickets, documentos en Drive

4.3 La ventaja que se acumula

Por qué esto no envejece

Cada vez que Anthropic lanza una versión más capaz de Claude — con mejor razonamiento, mayor ventana de contexto, nuevas capacidades — el sistema mejora sin tocar código.

Eso es lo que hace que este enfoque sea distinto a construir un sistema de orquestación propio: el razonamiento no está en el código, está en el modelo. Y el modelo mejora solo.

Las firmas que empiecen a operar de esta manera hoy van a desarrollar una intuición colectiva sobre cómo usar la IA bien. Esa intuición no se transfiere fácilmente. Es una ventaja que se acumula.

Parte V: Principios que Evitan Repetir los Errores del Pasado

Cualquier implementación de esta estrategia debería estar guiada por los siguientes principios. Son la diferencia entre una lancha ágil y un sistema que, con el tiempo, se vuelve tan pesado como aquello que intentaba reemplazar.

1	El servidor de herramientas no razona Cada herramienta hace una sola cosa: ir a buscar datos y devolverlos. La decisión de qué buscar, cuándo buscarlo y qué hacer con los resultados es de Claude, no del código. Si hay lógica de negocio en el servidor, es el primer paso hacia la complejidad que queremos evitar.
2	El proceso vive en el lenguaje, no en el código Las instrucciones de cómo crear un BRSD, un WSR o cualquier otro entregable viven en el system prompt del Claude Project, no en Python. Cambiar el proceso es editar un texto. El equipo de consultoría puede ajustarlo sin involucrar a desarrollo.
3	Aislamiento estricto entre proyectos Toda consulta de datos lleva el identificador del proyecto. El sistema está diseñado para que sea imposible, por construcción, que la información de un cliente sea visible en el contexto de otro.
4	Sin estado propio que mantener El servidor de herramientas no guarda información. No hay base de datos que mantener sincronizada, no hay espejo de datos. La información vive donde ya vive — en Wrike, en Gmail, en Drive.
5	Empezar pequeño, crecer con evidencia La primera versión resuelve un solo problema para un solo equipo piloto. Las decisiones de expansión se toman con base en lo que el uso real muestra, no en suposiciones previas.

Parte VI: Preguntas Abiertas y Próximos Pasos

6.1 Lo que todavía no sabemos

Este documento es una hipótesis de trabajo, no una conclusión. Las siguientes preguntas son las que deben responderse antes de comprometer recursos significativos:

- ¿Qué tan bueno es Claude en la práctica al generar los entregables específicos de consultoría de la firma, una vez que tiene el contexto correcto? Esto solo se puede responder con una prueba real.
- ¿Qué tan completo y útil es el contexto que las herramientas pueden traer de las fuentes actuales (Wrike, Zoom, Gmail, Drive)? ¿Hay información importante que vive en otros sistemas?
- ¿Cuál es el nivel de adopción real de los consultores cuando tienen la herramienta disponible? ¿La usan? ¿Les genera valor percibido?
- ¿Hay casos de uso donde la complejidad de Cerebro sí es necesaria y donde una solución liviana no alcanza?

6.2 El camino más corto hacia una respuesta

La forma más rápida de responder estas preguntas no es seguir investigando en abstracto — es construir la versión mínima del sistema y ponerla en manos de un consultor real con un proyecto real.

Un piloto así requeriría tres definiciones previas:

- Elegir el cliente y el proyecto que servirán como caso de prueba.
- Definir el entregable específico que el consultor producirá con el sistema (BRSD, WSR u otro).
- Asegurar el acceso técnico a las fuentes de datos relevantes (Wrike, y al menos una de Gmail o Drive).

Lo que se aprenda de ese piloto es lo que debería guiar las decisiones de inversión y escala.

La pregunta que desbloquea todo

¿Cuál es el proyecto piloto y quién es el consultor que va a probarlo?

Esa decisión no es técnica. Es la que convierte esta exploración en un experimento con resultados reales.

El barco grande sigue en construcción.
Pero el mar no espera.